Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Secuencias y Transformer

Módulo 14

El Transformer completo

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 14: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$\mathcal{V}$, $V = \lvert\mathcal{V}\rvert$	vocabulario y su tamaño
n, n_{ctx}	longitud de la secuencia y longitud máxima de contexto
d, h, $d_k = d/h$	dimensión del modelo, cabezas y dimensión por cabeza
L, d_{ff}	número de bloques y dimensión de la capa intermedia del MLP
$H^\ell \in \mathbb{R}^{n\times d}$, h_t^ℓ	flujo residual tras el bloque ℓ y su fila t
$W_E \in \mathbb{R}^{V\times d}$, $W_U \in \mathbb{R}^{d\times V}$	matrices de embedding y de *unembedding*
W_Q, W_K, W_V, W_O	proyecciones de la atención ($d \times d$)
$W_g, W_u \in \mathbb{R}^{d\times d_{\text{ff}}}$	matrices de puerta y de subida de SwiGLU
$W_{\text{d}} \in \mathbb{R}^{d_{\text{ff}}\times d}$	matriz de bajada de SwiGLU
$M \in \{0, -\infty\}^{n\times n}$	máscara causal
$\sigma(x)$, $\text{SiLU}(x) = x\,\sigma(x)$	sigmoide logística y función SiLU
$R(\alpha)$, R_m	rotación de ángulo α; matriz de RoPE en la posición m
$\text{PE}(p)$	codificación sinusoidal de la posición p
N, D, C	número de parámetros, de tokens de entrenamiento y de FLOPs
P	matriz de permutación

Los vectores de tokens son filas: una secuencia es una matriz $X \in \mathbb{R}^{n\times d}$ y una capa lineal actúa como $X \mapsto XW$, como en [8] y en el código de PyTorch. Los números grandes se escriben con espacio fino como separador de miles ($6\,738\,415\,616$).

Índice

Sobre este resumen **ii**

1 Arquitectura del Transformer decoder-only **1**
1.1 Entrada: tokens y embeddings . . . 1
1.2 El bloque decoder-only . . . 1
1.3 Causalidad . . . 2
1.4 Salida: unembedding y embeddings atados . . . 2

2 Flujo residual y posición de la normalización **4**
2.1 El flujo residual . . . 4
2.2 Pre-norm frente a post-norm . . . 5
2.3 El precio de pre-norm: crecimiento del flujo . . . 5

3 La subcapa MLP: GELU y SwiGLU **7**
3.1 El MLP posición a posición . . . 7
3.2 Activaciones suaves: GELU y SiLU . . . 7
3.3 Unidades lineales con puerta y SwiGLU . . . 8

4 Codificación de la posición: sinusoidal y RoPE **9**
4.1 La atención no ve el orden . . . 9
4.2 Codificación sinusoidal y posiciones aprendidas . . . 9
4.3 Codificación rotatoria (RoPE) . . . 10

5 Contar parámetros y memoria **12**
5.1 Parámetros de un bloque . . . 12
5.2 Parámetros del modelo completo . . . 12
5.3 Memoria . . . 13

6 Contar el cómputo: FLOPs y la regla $6ND$ **14**
6.1 El coste de un producto de matrices . . . 14
6.2 La pasada hacia delante . . . 14
6.3 La pasada hacia atrás y la regla $6ND$ . . . 15

Bibliografía **16**

Tema 1

Arquitectura del Transformer decoder-only

Este tema describe la arquitectura completa de un Transformer decoder-only: la entrada (tokens y embeddings), la pila de bloques y la capa de salida que produce los logits. Se demuestran dos propiedades estructurales: cada bloque conserva la forma $n \times d$ y la salida en la posición t solo depende de los tokens $x_{1:t}$. Se sigue [8, §7.2–§7.5], [1, §12.1.7 y §12.3.1], el artículo original [17, §3] y la descripción de la práctica 1 de CS336 [6, §3.1 y §3.4].

1.1 Entrada: tokens y embeddings

Un tokenizador (módulo 12) transforma el texto en una secuencia de enteros $x_{1:n} = (x_1, \dots, x_n)$ con $x_t \in \{1, \dots, V\}$.

Definición 1.1 (Embedding de tokens). Sea $W_E \in \mathbb{R}^{V \times d}$ una matriz de parámetros cuya fila j, $e_j \in \mathbb{R}^d$, es el *embedding* del token j. La entrada de la red es la matriz $H^0 \in \mathbb{R}^{n \times d}$ cuya fila t es

$$h_t^0 = e_{x_t} \ (+\, p_t),$$

donde $p_t \in \mathbb{R}^d$ es un vector de posición que solo se suma en los modelos con posiciones absolutas (tema 4). Equivalentemente, $H^0 = OW_E$, siendo $O \in \{0,1\}^{n \times V}$ la matriz cuyas filas son las codificaciones *one-hot* de los tokens.

La forma matricial OW_E muestra que el embedding es una capa lineal sin sesgo aplicada a vectores *one-hot*; en la práctica se implementa como una indexación de filas, que no hace ninguna multiplicación [8, §7.4].

1.2 El bloque decoder-only

Se recuerda la atención del módulo 13. Para una matriz $Z \in \mathbb{R}^{n \times d}$ y h cabezas de dimensión $d_k = d/h$, la cabeza i calcula $Q_i = ZW_Q^{(i)}$, $K_i = ZW_K^{(i)}$, $V_i = ZW_V^{(i)}$, con $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d \times d_k}$, y

$$\mathrm{head}_i(Z) = \mathrm{softmax}\Big(\frac{Q_i K_i^\top}{\sqrt{d_k}} + M\Big) V_i, \qquad M_{tu} = \begin{cases} 0, & u \le t, \\ -\infty, & u > t, \end{cases}$$

donde la softmax se aplica por filas. La atención multicabeza concatena las cabezas y proyecta: $\mathrm{MHA}(Z) = [\mathrm{head}_1(Z) \ \cdots \ \mathrm{head}_h(Z)]\, W_O$, con $W_O \in \mathbb{R}^{d \times d}$. Agrupando por columnas las h matrices $W_Q^{(i)}$ se obtiene una sola matriz $W_Q \in \mathbb{R}^{d \times d}$, y lo mismo para W_K y W_V.

Definición 1.2 (Bloque pre-norm). Sean $\mathrm{Norm}_1, \mathrm{Norm}_2 : \mathbb{R}^d \to \mathbb{R}^d$ dos normalizaciones (RMSNorm en LLaMA, LayerNorm en GPT-2), aplicadas fila a fila, y sea $\mathrm{MLP} : \mathbb{R}^d \to \mathbb{R}^d$

un perceptrón aplicado fila a fila (tema 3). El *bloque decoder-only pre-norm* es la aplicación $B : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$, $H \mapsto H'$, dada por

$$A = H + \mathrm{MHA}\big(\mathrm{Norm}_1(H)\big), \qquad H' = A + \mathrm{MLP}\big(\mathrm{Norm}_2(A)\big).$$

Definición 1.3 (Transformer decoder-only). Un *Transformer decoder-only* de L capas es la composición

$$H^\ell = B_\ell\big(H^{\ell-1}\big) \quad (\ell = 1, \dots, L), \qquad Z = \mathrm{Norm}_f\big(H^L\big)\, W_U \in \mathbb{R}^{n\times V},$$

donde H^0 es la entrada de la definición 1.1, cada B_ℓ es un bloque con parámetros propios, Norm_f es la *normalización final* y $W_U \in \mathbb{R}^{d\times V}$ es la matriz de *unembedding*. La fila t de Z son los logits de la distribución del token siguiente: $P(x_{t+1} = j \mid x_{1:t}) = \mathrm{softmax}(z_t)_j$.

La figura 1.1 representa la arquitectura. Las dos subcapas tienen papeles complementarios: la atención es la única operación que combina filas distintas, es decir, que mezcla información *entre* tokens; las normalizaciones y el MLP actúan sobre cada fila por separado [8, §7.2].

Proposición 1.4 (Conservación de la forma). *Si $H \in \mathbb{R}^{n\times d}$, entonces $B(H) \in \mathbb{R}^{n\times d}$. En consecuencia, todos los flujos $H^0, \dots, H^L$ tienen forma $n \times d$, y solo la entrada (n enteros $\to n \times d$) y la salida ($n \times d \to n \times V$) cambian de dimensión.*

La proposición 1.4 es la razón por la que los bloques pueden apilarse en cualquier número: L es un hiperparámetro libre, que va de 12 en GPT-2 *small* a 80 en Llama 2 70B [12, 16].

1.3 Causalidad

Proposición 1.5 (Causalidad del decoder). *En un Transformer decoder-only, la fila t de H^ℓ, para todo ℓ, y la fila t de los logits Z solo dependen de los tokens $x_1, \dots, x_t$.*

Corolario 1.6 (Una pasada, n predicciones). *Una sola evaluación del Transformer sobre $x_{1:n}$ proporciona las n distribuciones $P(\cdot \mid x_{1:t})$, $t = 1, \dots, n$. La pérdida de entrenamiento de una secuencia, la entropía cruzada media del módulo 1,*

$$\mathcal{L} = -\frac{1}{n-1} \sum_{t=1}^{n-1} \ln \mathrm{softmax}(z_t)_{x_{t+1}},$$

se calcula, por tanto, con una única pasada hacia delante y una única pasada hacia atrás.

Esta propiedad es la que hace eficiente el entrenamiento de los Transformer frente a las redes recurrentes: las n posiciones se procesan en paralelo como un producto de matrices [8, §7.3]. En la generación, en cambio, los tokens se producen de uno en uno, y la proposición 1.5 permite reutilizar las claves y los valores ya calculados (la caché KV del módulo 19).

1.4 Salida: unembedding y embeddings atados

La capa de salida transforma cada vector $h \in \mathbb{R}^d$ en V logits mediante $z = hW_U$. Si se escribe $u_j \in \mathbb{R}^d$ para la columna j de W_U, el logit del token j es el producto escalar $z_j = h \cdot u_j$: el modelo puntúa cada token por la alineación de h con un vector *plantilla* de ese token [8, §7.5].

Definición 1.7 (Embeddings atados). Un Transformer tiene los *embeddings atados* (*weight tying*) si $W_U = W_E^\top$, es decir, si la plantilla de salida de cada token coincide con su embedding de entrada: $z_j = h \cdot e_j$.

Press y Wolf [11] mostraron que atar los embeddings reduce la perplejidad de varios modelos de lenguaje y el número de parámetros. El Transformer original comparte la misma matriz entre los embeddings y la capa previa a la softmax, y multiplica los embeddings por $\sqrt{d}$ [17, §3.4]. GPT-2 también ata los embeddings; LLaMA y Llama 2 usan dos matrices independientes [10].

Proposición 1.8 (Ahorro del atado). *Atar los embeddings reduce el número de parámetros en exactamente Vd. Para un vector h fijo, el token de mayor logit es el j que maximiza $\|e_j\| \cos \angle(h, e_j)$.*

Cuadro 1.1: Elecciones de arquitectura de tres modelos con detalles públicos.

Elemento	Transformer (2017)	GPT-2 (2019)	LLaMA / Llama 2 (2023)
Normalización	LayerNorm, post-norm	LayerNorm, pre-norm	RMSNorm, pre-norm
MLP	ReLU, $d_{\text{ff}} = 4d$	GELU, $d_{\text{ff}} = 4d$	SwiGLU, $d_{\text{ff}} \approx \frac{8}{3}d$
Posición	sinusoidal (sumada)	aprendida (sumada)	RoPE (en q y k)
Sesgos	sí	sí	no
Embeddings	atados	atados	sin atar

Tema 2

Flujo residual y posición de la normalización

En el módulo 11 se estudiaron las conexiones residuales y la normalización para una red genérica. Este tema las aplica al bloque del tema 1: desarrolla el flujo residual de un Transformer como suma de $2L$ aportaciones, compara las variantes pre-norm y post-norm y justifica la normalización final y la inicialización escalada. Se sigue [8, §7.2], la discusión de la práctica 1 de CS336 [6, §3.4], el análisis de Xiong *et al.* [19], la §2.3 de [12] y la interpretación de Elhage *et al.* [4].

2.1 El flujo residual

Para simplificar la notación, se numeran las $2L$ subcapas del Transformer en el orden en que se aplican: la subcapa $k = 2\ell - 1$ es la atención del bloque ℓ y la subcapa $k = 2\ell$ es su MLP. Sea F_k la función que calcula la subcapa k *incluida su normalización de entrada*: $F_{2\ell-1} = \mathrm{MHA} \circ \mathrm{Norm}_1$ y $F_{2\ell} = \mathrm{MLP} \circ \mathrm{Norm}_2$.

Definición 2.1 (Flujo residual). El *flujo residual* es la sucesión de matrices $R^0 = H^0$ y $R^k = R^{k-1} + F_k(R^{k-1})$, $k = 1, \ldots, 2L$. Se cumple $R^{2\ell} = H^\ell$ y $R^{2\ell-1} = A^\ell$, la matriz intermedia del bloque ℓ. La fila t de R^k es el vector del flujo del token t.

Proposición 2.2 (Desarrollo telescópico). *En un Transformer pre-norm,*

$$H^L = H^0 + \sum_{k=1}^{2L} F_k\big(R^{k-1}\big).$$

La proposición 2.2 sustenta la lectura del flujo como un *canal de comunicación* [4]: ninguna subcapa sobrescribe el flujo; cada una lee de él (tras normalizar) y le *suma* su salida. De ahí se siguen tres consecuencias.

(i) Una subcapa puede escribir en una dirección de $\mathbb{R}^d$ que lee otra subcapa muchas capas después. Esta descomposición es la base de la interpretabilidad mecanicista (módulo 23).

(ii) Toda subcapa debe producir vectores de $\mathbb{R}^d$, pues su salida se suma al flujo. Esa es la función de W_O en la atención y de W_{d} en el MLP.

(iii) Si una subcapa aprende a escribir casi cero, el modelo se comporta como uno con una subcapa menos.

Proposición 2.4 (Jacobiano del flujo). *Sea J_k el jacobiano de F_k evaluado en R^{k-1}, con todas las matrices vectorizadas. Entonces*

$$\frac{\partial H^L}{\partial H^0} = \prod_{k=2L}^{1} (I + J_k) = I + \sum_{k=1}^{2L} J_k + \sum_{k>k'} J_k J_{k'} + \cdots + J_{2L} \cdots J_1,$$

donde el producto se ordena de izquierda a derecha con k decreciente.

El término I del desarrollo es el *camino identidad*: el gradiente de la pérdida respecto de H^L llega a H^0 sumado, sin atravesar ninguna normalización ni producto de jacobianos. Es la versión para el Transformer del teorema del jacobiano residual del módulo 11.

2.2 Pre-norm frente a post-norm

Definición 2.5 (Post-norm). El *bloque post-norm* del Transformer original [17, §3.1] aplica la normalización después de la suma residual:

$$A = \mathrm{Norm}_1\big(H + \mathrm{MHA}(H)\big), \qquad H' = \mathrm{Norm}_2\big(A + \mathrm{MLP}(A)\big).$$

Proposición 2.6 (No hay camino identidad en post-norm). *En un Transformer post-norm con normalización RMSNorm sin ganancia,* $\mathrm{Norm}(x) = x/\,\mathrm{RMS}(x)$*, se cumple* $\mathrm{RMS}(h_t^\ell) = 1$ *para toda fila t y toda capa $\ell \geq 1$, y el jacobiano de la salida respecto de la entrada es*

$$\frac{\partial H^L}{\partial H^0} = \prod_{k=2L}^{1} N_k\,(I + J_k),$$

donde N_k es el jacobiano de la normalización k. Cada N_k es singular.

En post-norm, toda la señal, y todo el gradiente, atraviesa $2L$ normalizaciones; no existe un término I aislado como en la proposición 2.4. Xiong *et al.* [19, §3.3, teorema 1] cuantifican la diferencia en la inicialización: bajo hipótesis de concentración de las normas, la norma del gradiente de los pesos de la última capa es del orden de $d\sqrt{\ln d}$ en post-norm, sin dependencia de L, y del orden de $d\sqrt{\ln d/L}$ en pre-norm. Gradientes grandes en las capas altas obligan a un calentamiento (*warmup*) largo de la tasa de aprendizaje en post-norm; pre-norm se entrena de forma estable con un calentamiento corto o nulo.

Cuadro 2.1: Comparación de las dos posiciones de la normalización.

Aspecto	Post-norm (2017)	Pre-norm (GPT-2, LLaMA)
Ecuación	$\mathrm{Norm}(x + F(x))$	$x + F(\mathrm{Norm}(x))$
Camino identidad	no (proposición 2.6)	sí (proposición 2.4)
Norma del flujo	fija	crece con la profundidad
Calentamiento	imprescindible	tolerante
Normalización final	innecesaria	necesaria

2.3 El precio de pre-norm: crecimiento del flujo

Proposición 2.7 (Crecimiento del flujo en un Transformer). *Supóngase que, en la inicialización, las aportaciones $F_k(R^{k-1})_t$ de una fila son vectores aleatorios de media nula, incorrelados entre sí y con h_t^0, con $\mathbb{E}\|F_k(R^{k-1})_t\|^2 = s^2$. Entonces*

$$\mathbb{E}\|h_t^L\|^2 = \mathbb{E}\|h_t^0\|^2 + 2L\,s^2.$$

La proposición 2.7 tiene dos consecuencias prácticas. Primera: como la norma del flujo crece, el vector h_t^L debe normalizarse antes del unembedding; de lo contrario, la escala de los logits

dependería de la profundidad. Es la normalización final Norm_f de la definición 1.3, que GPT-2 introdujo junto con la posición pre-norm [12, §2.3] y que Jurafsky y Martin señalan como la capa adicional que exige pre-norm [8, §7.2.3]. Segunda: GPT-2 multiplica en la inicialización los pesos de las capas que escriben en el flujo (W_O y la última matriz del MLP) por $1/\sqrt{N}$, con $N = 2L$ el número de capas residuales; así cada s^2 se divide por $2L$ y la norma esperada del flujo final queda acotada independientemente de la profundidad.

Tema 3

La subcapa MLP: GELU y SwiGLU

La segunda subcapa de cada bloque es un perceptrón de una capa oculta que se aplica por separado a cada token. Contiene unos dos tercios de los parámetros del modelo. Este tema define el MLP clásico del Transformer original y de GPT-2, las unidades lineales con puerta y la variante SwiGLU de LLaMA, demuestra las propiedades de la activación SiLU y justifica la elección $d_{\text{ff}} \approx \frac{8}{3}d$. Se sigue [17, §3.3], [20, §11.7.2], [8, §7.2.1], el artículo de Shazeer [13] y la §3.4.2 de la práctica de CS336 [6].

3.1 El MLP posición a posición

Definición 3.1 (MLP posición a posición). Sea $\phi : \mathbb{R} \to \mathbb{R}$ una función de activación aplicada componente a componente. El *MLP posición a posición* con dimensión intermedia d_{ff} es

$$\text{FFN}(x) = \phi\big(xW_1 + b_1\big)W_2 + b_2, \qquad W_1 \in \mathbb{R}^{d\times d_{\text{ff}}},\ W_2 \in \mathbb{R}^{d_{\text{ff}}\times d},$$

aplicado a cada fila $x \in \mathbb{R}^d$ de la matriz de entrada con los mismos parámetros.

El Transformer original usa $\phi = \text{ReLU}$, $d = 512$ y $d_{\text{ff}} = 2048 = 4d$ [17, §3.3]; GPT-2 usa la misma proporción con la activación GELU. Que el MLP actúe fila a fila tiene una consecuencia estructural.

Lema 3.2 (Las operaciones fila a fila conmutan con las permutaciones). *Sea $f : \mathbb{R}^d \to \mathbb{R}^m$ y sea $\hat{f} : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times m}$ la aplicación que actúa como f sobre cada fila. Para toda matriz de permutación $P \in \mathbb{R}^{n\times n}$, $\hat{f}(PX) = P\,\hat{f}(X)$.*

El lema 3.2 se aplica al MLP, a las normalizaciones y a las proyecciones lineales. Por eso la atención es la única subcapa sensible al orden de los tokens, y por eso hace falta codificar la posición (tema 4).

3.2 Activaciones suaves: GELU y SiLU

Definición 3.3 (GELU y SiLU). Sean Φ la función de distribución de la normal estándar y $\sigma(x) = 1/(1 + e^{-x})$ la sigmoide. Se definen

$$\text{GELU}(x) = x\,\Phi(x), \qquad \text{SiLU}(x) = x\,\sigma(x).$$

La función SiLU también se llama *Swish* con parámetro 1 [13].

Ambas multiplican x por un número entre 0 y 1 que crece con x: son versiones suaves de $\text{ReLU}(x) = x \cdot \mathbf{1}[x > 0]$, en las que el interruptor $\mathbf{1}[x > 0]$ se sustituye por una función de distribución [7].

Proposición 3.4 (Propiedades de SiLU). *La función SiLU cumple:*

(i) $\mathrm{SiLU}(x) - \mathrm{ReLU}(x) \to 0$ *cuando* $x \to \pm\infty$*;*

(ii) $\mathrm{SiLU}'(x) = \sigma(x)\big(1 + x(1 - \sigma(x))\big)$*; en particular,* $\mathrm{SiLU}(0) = 0$ *y* $\mathrm{SiLU}'(0) = 1/2$*;*

(iii) tiene un único punto crítico $x^* < 0$*, que es un mínimo global, con* $\mathrm{SiLU}(x^*) = x^* + 1$*. Numéricamente,* $x^* \approx -1{,}2785$ *y* $\mathrm{SiLU}(x^*) \approx -0{,}2785$*.*

A diferencia de ReLU, SiLU es derivable en todo punto, no es monótona y deja pasar valores negativos pequeños, lo que evita las neuronas cuyo gradiente es nulo en todo el semieje negativo.

3.3 Unidades lineales con puerta y SwiGLU

Definición 3.5 (SwiGLU). Sean $W_g, W_u \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$ y $W_{\mathrm{d}} \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$. La subcapa *SwiGLU* es

$$\mathrm{SwiGLU}(x) = \big(\mathrm{SiLU}(xW_g) \odot xW_u\big)\, W_{\mathrm{d}},$$

donde $\odot$ es el producto componente a componente. La rama xW_g es la *puerta* y xW_u la *rama lineal*.

Las unidades lineales con puerta (GLU) de Dauphin *et al.* [3] multiplican una proyección lineal por una puerta sigmoide; Shazeer [13, §2] probó varias funciones en la puerta y encontró que la variante con Swish mejoraba la perplejidad del Transformer. Las componentes de la capa intermedia ya no son $\phi(\text{lineal})$, sino productos de dos formas lineales del token: la puerta decide cuánto de la rama lineal se transmite.

Proposición 3.6 (Presupuesto de parámetros). *Sin sesgos, el MLP de la definición 3.1 tiene* $2d\,d_{\mathrm{ff}}$ *parámetros, y SwiGLU tiene* $3d\,d_{\mathrm{ff}}$*. Con* $d_{\mathrm{ff}} = 4d$*, el primero tiene* $8d^2$ *parámetros; SwiGLU tiene el mismo número si y solo si* $d_{\mathrm{ff}} = \frac{8}{3}d$*.*

El mismo argumento iguala el cómputo, porque el coste de cada producto es proporcional al número de pesos (tema 6). Shazeer [13, §2] reduce d_{ff} en un factor $2/3$ por esta razón: en sus experimentos, el MLP de dos matrices usa $d_{\mathrm{ff}} = 3072$ y las variantes con puerta $d_{\mathrm{ff}} = 2048$.

Tema 4

Codificación de la posición: sinusoidal y RoPE

Sin información de posición, la atención trata la secuencia como un conjunto. Este tema demuestra esa propiedad, define la codificación sinusoidal del Transformer original y las posiciones aprendidas de GPT-2, y estudia con detalle la codificación rotatoria RoPE de LLaMA, cuya puntuación de atención solo depende de la distancia entre tokens. Se sigue [17, §3.5], [20, §11.6.3], [1, §12.1.9], [8, §7.4], el artículo de Su *et al.* [14, §3.2] y las secciones 2 y 3 de la práctica 4 de CS224N [2].

4.1 La atención no ve el orden

Proposición 4.1 (Equivariancia por permutaciones). *Sea*

$$f(X) = \mathrm{softmax}\big(XW_Q(XW_K)^\top/\sqrt{d_k}\big)XW_V$$

la autoatención sin máscara*, con la softmax por filas. Para toda matriz de permutación P, $f(PX) = P\,f(X)$.*

Junto con el lema 3.2, la proposición 4.1 implica que un Transformer sin máscara y sin posiciones es equivariante: barajar los tokens de la entrada solo baraja las filas de la salida, de modo que «el perro muerde al hombre» y «el hombre muerde al perro» producen los mismos vectores por palabra. La máscara causal rompe la simetría (ejercicio 1.6), y la práctica de CS336 recuerda que un decoder causal puede, en principio, inferir posiciones sin codificación explícita [6, §7.3]; aun así, todos los modelos de la cuadro 1.1 codifican la posición de forma explícita.

4.2 Codificación sinusoidal y posiciones aprendidas

Definición 4.2 (Codificación sinusoidal). Sea d par y $\omega_i = 10000^{-2i/d}$ para $i = 0, \dots, d/2-1$. La *codificación sinusoidal* de la posición $p \in \mathbb{N}$ es el vector $\mathrm{PE}(p) \in \mathbb{R}^d$ con componentes

$$\mathrm{PE}(p)_{2i} = \sin(p\,\omega_i), \qquad \mathrm{PE}(p)_{2i+1} = \cos(p\,\omega_i),$$

que se suma al embedding del token: $h_t^0 = e_{x_t} + \mathrm{PE}(t)$ [17, §3.5].

Cada par $(2i, 2i+1)$ es un punto que gira sobre la circunferencia unidad con frecuencia angular ω_i; las frecuencias decrecen geométricamente desde $\omega_0 = 1$ hasta $\omega_{d/2-1} \approx 10^{-4}$. Es como un reloj con $d/2$ agujas que giran a velocidades distintas [20, §11.6.3.1].

Teorema 4.4 (Desplazamiento lineal). *Para cada $k \in \mathbb{Z}$ existe una matriz ortogonal $M_k \in \mathbb{R}^{d\times d}$, que no depende de p, tal que $\mathrm{PE}(p+k) = M_k\,\mathrm{PE}(p)$ para todo p.*

El teorema 4.4 es la afirmación de Vaswani *et al.* [17, §3.5] de que $\mathrm{PE}(p+k)$ es una función lineal de $\mathrm{PE}(p)$ para cada desplazamiento fijo, y la propiedad de posición relativa de [20, §11.6.3.2]. La esperanza era que la red aprendiera a atender por posiciones relativas mediante esa aplicación lineal.

Definición 4.5 (Posiciones aprendidas). Un modelo con *posiciones aprendidas* añade una matriz de parámetros $W_P \in \mathbb{R}^{n_{\mathrm{ctx}}\times d}$ y suma su fila t al embedding del token t. Es la opción de GPT-2 [12]. Aporta $n_{\mathrm{ctx}}d$ parámetros y no define ninguna codificación para posiciones $t > n_{\mathrm{ctx}}$.

4.3 Codificación rotatoria (RoPE)

En lugar de sumar la posición al embedding, RoPE la introduce en cada capa, girando las consultas y las claves [14]. Sea

$$R(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}$$

la rotación plana de ángulo α.

Lema 4.6 (Rotaciones planas). *Para todos $a, b \in \mathbb{R}$: (i) $R(a)R(b) = R(a+b)$; (ii) $R(a)^\top = R(-a)$; (iii) en consecuencia, $R(a)^\top R(b) = R(b-a)$ y $R(a)$ es ortogonal.*

Definición 4.7 (RoPE). Sea d_k par y $\theta_i = 10000^{-2i/d_k}$ para $i = 0, \ldots, d_k/2-1$. Para cada posición m se define la matriz diagonal por bloques

$$R_m = \mathrm{diag}\big(R(m\theta_0),\, R(m\theta_1),\, \ldots,\, R(m\theta_{d_k/2-1})\big) \in \mathbb{R}^{d_k\times d_k}.$$

RoPE sustituye, en cada cabeza y cada capa, la consulta q_m del token m por $\tilde{q}_m = R_m q_m$ y la clave k_n del token n por $\tilde{k}_n = R_n k_n$. Los valores v_n no se modifican.

Teorema 4.8 (RoPE es relativa). *Para todos $q, k \in \mathbb{R}^{d_k}$ y todas las posiciones m, n,*

$$(R_m q)^\top (R_n k) = q^\top R_{n-m}\, k.$$

En particular, la puntuación de atención entre el token m y el token n solo depende de las posiciones a través de la diferencia $n-m$.

Corolario 4.9. *RoPE conserva las normas: $\|R_m q\| = \|q\|$. Solo modifica el ángulo entre la consulta y la clave.*

Para un solo par ($d_k = 2$), si $\varphi = (n-m)\theta$, el desarrollo de $q^\top R(\varphi)k$ es

$$q^\top R(\varphi)\, k = (q\cdot k)\cos\varphi - (q_1k_2 - q_2k_1)\sin\varphi. \tag{4.1}$$

Proposición 4.11 (Forma compleja). *Identifíquese cada par (a,b) con el número complejo $z = a+ib$. Entonces la rotación $R(\alpha)$ equivale a multiplicar por $e^{i\alpha}$, y, con $\langle u, v\rangle = \mathrm{Re}(u\bar{v})$, el producto escalar de dos pares rotados es*

$$\big\langle z_1 e^{im\theta},\, z_2 e^{in\theta}\big\rangle = \mathrm{Re}\big(z_1\bar{z}_2\, e^{i(m-n)\theta}\big).$$

Cuadro 4.1: Puntuaciones $(R_m q)^\top (R_n k)$ del ejemplo 4.10.

m (consulta)	n (clave)	$n - m$	Puntuación
3	1	-2	$-1{,}830$
5	3	-2	$-1{,}830$
2	0	-2	$-1{,}830$
1	3	$+2$	6,830

La forma compleja es la que usan el artículo original [14, §3.2.1] y el código de LLaMA [10], que agrupa las componentes consecutivas de q y k en números complejos. La fase $e^{i(m-n)\theta}$ hace el papel del desfase entre dos osciladores de la misma frecuencia: solo importa la diferencia de fases, como en la interferencia de dos ondas.

Cuadro 4.2: Comparación de las codificaciones de posición. La propiedad relativa es la del teorema 4.4 para la sinusoidal y la del teorema 4.8 para RoPE.

Método	Dónde actúa	Parámetros	Relativa	Modelos
Sinusoidal	suma al embedding	0	indirecta	Transformer
Aprendida	suma al embedding	$n_{\text{ctx}} d$	no	GPT-2, GPT-3
RoPE	gira q y k en cada capa	0	exacta	LLaMA, Qwen

Tema 5

Contar parámetros y memoria

A partir de la configuración de un Transformer (V, d, L, h, d_{ff}) puede calcularse sin ejecutar nada cuántos parámetros tiene y cuánta memoria necesita para la inferencia y para el entrenamiento. Este tema deduce la fórmula exacta para la arquitectura de LLaMA, la aproximación $N \approx 12Ld^2$ de Kaplan *et al.* [9, §2.1] y el coste en bytes por parámetro del entrenamiento con Adam en precisión mixta. Se sigue también [8, §7.1.1 y ejercicio 7.1] y el problema `transformer_accounting` de CS336 [6].

5.1 Parámetros de un bloque

Proposición 5.1 (La atención no depende del número de cabezas). *Sin sesgos, la atención multicabeza con h cabezas de dimensión $d_k = d/h$ tiene $4d^2$ parámetros, cualquiera que sea h.*

Proposición 5.2 (Parámetros de un bloque). *(i) Un bloque de tipo LLaMA (RMSNorm con ganancia, SwiGLU, sin sesgos) tiene $4d^2 + 3d\,d_{\text{ff}} + 2d$ parámetros.*

(ii) Un bloque de tipo GPT-2 (LayerNorm con ganancia y sesgo, MLP con $d_{\text{ff}} = 4d$, sesgos en todas las capas lineales) tiene $12d^2 + 13d$ parámetros.

(iii) Con $d_{\text{ff}} = 4d$ en un MLP de dos matrices o $d_{\text{ff}} = \frac{8}{3}d$ en SwiGLU, la parte matricial del bloque tiene $12d^2$ parámetros: $4d^2$ en la atención y $8d^2$ en el MLP.

5.2 Parámetros del modelo completo

Teorema 5.3 (Recuento exacto). *Un Transformer de tipo LLaMA con embeddings sin atar tiene*

$$N = L\big(4d^2 + 3d\,d_{\text{ff}} + 2d\big) + 2Vd + d$$

parámetros. Con embeddings atados, el término $2Vd$ se sustituye por Vd; con posiciones aprendidas, se añaden $n_{\text{ctx}}d$.

Corolario 5.4 (Aproximación de Kaplan). *Si $d_{\text{ff}} = 4d$ (o $\frac{8}{3}d$ en SwiGLU), los parámetros sin contar embeddings ni normalizaciones son exactamente $12Ld^2$, y*

$$\frac{N}{12Ld^2} = 1 + \frac{V}{6Ld} + \frac{2L+1}{12Ld} \qquad \textit{(sin atar).}$$

Kaplan *et al.* [9, §2.1, ecuación (2.1) y tabla 1] definen N como el número de parámetros *sin embeddings*, $N \approx 2d\,L(2d_{\text{attn}} + d_{\text{ff}}) = 12Ld^2$, porque observaron que las leyes de escalado se expresan mejor con esa cantidad (módulo 16).

Cuadro 5.1: Parámetros de un bloque de LLaMA-2 7B.

Pieza	Fórmula	Parámetros
W_Q, W_K, W_V, W_O	$4d^2$	67 108 864
SwiGLU	$3d\,d_{\text{ff}}$	135 266 304
Dos RMSNorm	$2d$	8 192
Bloque		202 383 360

5.3 Memoria

Definición 5.6 (Memoria de los parámetros). Si cada parámetro se almacena con b bytes, los pesos ocupan Nb bytes. Se usan $b = 4$ en fp32, $b = 2$ en bf16 y fp16, $b = 1$ en int8 y $b = 1/2$ en int4 (módulos 11 y 19). En este módulo, 1 GB $= 10^9$ bytes y 1 GiB $= 2^{30}$ bytes.

Proposición 5.7 (Estado del entrenamiento con Adam en precisión mixta). *Entrenar con Adam en precisión mixta requiere, sin contar las activaciones,* 16 *bytes por parámetro.*

Cuadro 5.2: Memoria de LLaMA-2 7B ($N = 6\,738\,415\,616$) según el uso.

Uso	Bytes por parámetro	Memoria
Inferencia en bf16	2	13,5 GB
Inferencia en int4 (módulo 19)	0,5	3,4 GB
Pesos en fp32	4	27,0 GB
Entrenamiento, Adam en precisión mixta	16	107,8 GB

La cuadro 5.2 explica por qué un modelo de $7 \cdot 10^9$ parámetros cabe en una GPU de 24 GB para la inferencia en bf16, pero no para el entrenamiento: con 16 bytes por parámetro, en 24 GB solo caben los estados de un modelo de $1{,}5 \cdot 10^9$ parámetros, y aún habría que añadir las activaciones. Las técnicas ZeRO y FSDP (módulo 17) reparten estos 16 bytes entre varias GPU.

Tema 6

Contar el cómputo: FLOPs y la regla $6ND$

El coste de un Transformer se mide en operaciones de coma flotante (FLOPs). Casi todas proceden de productos de matrices, de modo que basta un único lema para contarlas. Este tema deduce el coste de una pasada hacia delante, unos $2N$ FLOPs por token más un término de atención proporcional al contexto, y el coste del entrenamiento, $C \approx 6ND$. Se sigue [9, §2.1], la sección de contabilidad de recursos de la práctica 1 de CS336 [6, §3.5] y los datos de entrenamiento de Llama 2 [16, §2.2.1 y tabla 2].

6.1 El coste de un producto de matrices

Definición 6.1 (FLOP). Un *FLOP* es una suma o una multiplicación en coma flotante. El *rendimiento* de un acelerador se mide en FLOP/s; su *aprovechamiento* (MFU, *model FLOPs utilization*) es el cociente entre los FLOP/s útiles que alcanza un entrenamiento y el máximo teórico del hardware [6, problema `adamw_accounting`].

Lema 6.2 (Regla $2mkp$). *El producto de una matriz $m \times k$ por una matriz $k \times p$ requiere $2mkp$ FLOPs, contando como $2k$ las operaciones de cada producto escalar de longitud k.*

6.2 La pasada hacia delante

Proposición 6.3 (Coste hacia delante). *Sea N_{mat} el número de parámetros de las matrices que multiplican al flujo (proyecciones de atención, MLP y unembedding). Una pasada hacia delante sobre una secuencia de n tokens requiere*

$$C_{\text{fwd}} = 2nN_{\text{mat}} + 4Ln^2d$$

FLOPs, más operaciones elemento a elemento (normalizaciones, softmax, activaciones, RoPE) de coste $O(Lnd + Ln^2h)$, que se desprecian. Por token, el coste es $2N_{\text{mat}} + 4Lnd \approx 2N + 4Lnd$.

Observación 6.4. Kaplan *et al.* [9, §2.1, ecuación (2.2)] escriben $C_{\text{fwd}} \approx 2N + 2Ln_{\text{ctx}}d$ por token. El factor es la mitad del de la proposición 6.3, lo que concuerda con tener en cuenta la máscara causal: el token t solo atiende a t posiciones, y el promedio de t sobre la secuencia es aproximadamente $n/2$. La práctica de CS336 cuenta los productos completos $n \times n$, como la proposición 6.3.

Corolario 6.5 (Contexto en el que la atención iguala a los pesos). *El término de atención por token, $4Lnd$, iguala a $2N$ cuando $n^* = N/(2Ld)$.*

Para LLaMA-2 7B, $n^* = 6\,738\,415\,616/(2 \cdot 32 \cdot 4096) \approx 25\,705$ tokens; con su contexto de 4096 tokens, la atención añade un 15,9 % al coste $2N$. Para contextos mucho mayores domina

el término cuadrático, lo que motiva FlashAttention (módulo 17) y la atención con ventana deslizante (módulo 18).

6.3 La pasada hacia atrás y la regla $6ND$

Teorema 6.6 (El retroceso cuesta el doble). *Sea $Y = XW$ una capa lineal con $X \in \mathbb{R}^{n\times a}$ y $W \in \mathbb{R}^{a\times b}$, y sea $G = \partial\mathcal{L}/\partial Y \in \mathbb{R}^{n\times b}$. La pasada hacia delante cuesta $2nab$ FLOPs y la pasada hacia atrás, que calcula*

$$\frac{\partial \mathcal{L}}{\partial X} = G W^{\top} \qquad y \qquad \frac{\partial \mathcal{L}}{\partial W} = X^{\top} G,$$

cuesta $4nab$ FLOPs.

Corolario 6.7 (Regla $6ND$). *Entrenar un Transformer de N parámetros con D tokens cuesta $C \approx 6ND$ FLOPs, despreciando el término de atención y las operaciones elemento a elemento.*

Kaplan *et al.* enuncian la misma estimación, $C \approx 6N$ FLOPs por token de entrenamiento [9, §2.1], que es la base de las leyes de escalado del módulo 16.

Bibliografía

[1] C. M. Bishop y H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Cap. 12, «Transformers» (§12.1 «Attention», en particular §12.1.7 «Transformer layers» y §12.1.9 «Positional encoding»; §12.3.1 «Decoder transformers»). https://www.bishopbook.com/

[2] CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2025. Práctica 4, «Self-Attention, Transformers, and Pretraining». https://web.stanford.edu/class/cs224n/assignments_w25/a4.pdf

[3] Y. N. Dauphin, A. Fan, M. Auli y D. Grangier. Language modeling with gated convolutional networks. En *International Conference on Machine Learning (ICML)*, 2017. https://arxiv.org/abs/1612.08083

[4] N. Elhage *et al.* A mathematical framework for Transformer circuits. *Transformer Circuits Thread*, Anthropic, 2021. https://transformer-circuits.pub/2021/framework/index.html

[5] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Informe técnico, 2024. https://arxiv.org/abs/2403.05530

[6] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 1, «Basics», versión 26.0.3. https://github.com/stanford-cs336/assignment1-basics

[7] D. Hendrycks y K. Gimpel. Gaussian error linear units (GELUs). Prepublicación, 2016. https://arxiv.org/abs/1606.08415

[8] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 7, «Transformers and Pretraining». https://web.stanford.edu/~jurafsky/slp3/

[9] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. Prepublicación, 2020. https://arxiv.org/abs/2001.08361

[10] Meta. Código de inferencia de Llama 2, archivo `llama/model.py`. Repositorio `meta-llama/llama`, 2023. https://github.com/meta-llama/llama

[11] O. Press y L. Wolf. Using the output embedding to improve language models. En *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2017. https://arxiv.org/abs/1608.05859

[12] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[13] N. Shazeer. GLU variants improve Transformer. Prepublicación, 2020. https://arxiv.org/abs/2002.05202

[14] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen e Y. Liu. RoFormer: Enhanced Transformer with rotary position embedding. Prepublicación, 2021. Publicado en *Neurocomputing*, 568, 2024. https://arxiv.org/abs/2104.09864

[15] H. Touvron *et al.* LLaMA: Open and efficient foundation language models. Prepublicación, 2023. https://arxiv.org/abs/2302.13971

[16] H. Touvron *et al.* Llama 2: Open foundation and fine-tuned chat models. Prepublicación, 2023. https://arxiv.org/abs/2307.09288

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[18] xAI. Grok-1: código y pesos publicados. 2024. https://github.com/xai-org/grok-1

[19] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang y T.-Y. Liu. On layer normalization in the Transformer architecture. En *International Conference on Machine Learning (ICML)*, 2020. https://arxiv.org/abs/2002.04745

[20] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 11, «Attention Mechanisms and Transformers» (§11.6 «Self-Attention and Positional Encoding» y §11.7 «The Transformer Architecture»). https://d2l.ai/

[21] B. Zhang y R. Sennrich. Root mean square layer normalization. En *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. https://arxiv.org/abs/1910.07467